

Building a Multi-Tenant Web App with Dancer2 and htmx

Jason Crome

What We're Covering Today

These two things address different concerns within the application...

...and work really well together ;-)

- **Dancer2** is well-suited for multi-tenant app development
- **htmx** makes modern frontend dev approachable without a SPA framework
- Together: a match made in [% favorite_afterlife_place %]

htmx — What Is It?

htmx gives you access to AJAX, CSS Transitions, WebSockets and Server Sent Events directly in HTML, using attributes.

```
1 <button hx-get="/dashboard"  
2         hx-target="#main-content"  
3         hx-swap="innerHTML">  
4   Say Hello  
5 </button>  
6  
7 <div id="main-content"></div>
```

- No build step. No framework. No `node_modules` rabbit hole.
- The server returns **HTML fragments**, not JSON.
- Your backend stays your backend.

The HX-* Headers

htmx adds request headers your server can inspect:

Header	Meaning
<code>HX-Request</code>	<code>true</code> ; this is an htmx request
<code>HX-Target</code>	ID of the element being targeted
<code>HX-Current-URL</code>	The browser's current URL
<code>HX-Boosted</code>	Request came from <code>hx-boost</code>
<code>HX-Trigger</code>	ID of the element that triggered the request

These are how you distinguish “full page load” from “htmx fragment request.”

How a Dancer2 App Renders Content

A typical Dancer2 route:

```
1 get '/dashboard' => sub {  
2   my $data = fetch_dashboard_data();  
3  
4   template 'dashboard', {  
5     user => logged_in_user(),  
6     stats => $data,  
7   };  
8 };
```

- `template()` wraps your view in the **layout**
- Layout provides the page chrome; `<html>`, `<head>`, `<body>`, nav, footer, etc.
- This is exactly what a browser expects on a full page load

Now, Introduce htmx

What happens when htmx makes a request to `/dashboard`?

It gets back the **entire page** - layout and all.

htmx swaps that full HTML blob into your `hx-target`... inside your existing page.



Test — What Do You Think Will Happen?

Imagine this link:

```
1 <a hx-get="/dashboard" hx-target="#main-content">Dashboard</a>
```

And `/dashboard` returns a full layout-wrapped page.

You get a page **inside** a page. Nav bars nested in nav bars. `<head>` tags in your `<body>`. It's turtles all the way down.

We need to teach Dancer2 to respond differently based on who's asking.

The `render()` Helper

A local sub that wraps `template()` and handles htmx requests:

```
1 sub render( $template, $args = {} ) {
2   croak "render(): No template specified!" unless $template;
3
4   my $options = { layout => config->{ layout } };
5   if ( request_header( 'HX-Request' ) ) {
6     $options = { layout => undef };
7     debug 'Request is ajax, not rendering a layout';
8   }
9
10  return template( $template, $args, $options );
11 }
```

Now routes just call `render()` instead of `template()` - one method to rule them all

```
1 get '/profile' => needs login => sub {
2   render( 'profile', { title => 'Profile' } );
3 };
```


Adding Multi-Tenancy

One app with many users, each with their own isolated database.

In this app, the **logged-in user is the tenant**.

- A shared **registry** DB holds user accounts
- Each user gets their own SQLite DB at **var/db/\${username}.db**
- Tenant connection is established on every request, torn down after

Plugins Do the Heavy Lifting

```
1 use Dancer2::Plugin::Database;      # database() keyword + connection mgmt
2 use Dancer2::Plugin::Auth::Tiny;    # needs login => route decoration
3 use Dancer2::Plugin::CryptPassphrase; # verify_password()
4 use Dancer2::Plugin::Syntax::ParamKeywords; # body_param(), query_params()
```

Dancer2's plugin ecosystem means you're not reinventing this stuff.

The **before** Hook

This is where the multitenancy magic lives:

```
1 hook 'before' => sub {
2   if ( my $username = session->read( 'user' ) ) {
3     my $user = database( 'registry' )->quick_select(
4       'users',
5       { username => $username }
6     );
7     if ( defined $user ) {
8       var user => $user;
9
10      my $tenant_dbh = database({
11        driver    => 'SQLite',
12        database => "var/db/${username}.db"
13      });
14      var tenant_dbh => $tenant_dbh;
15    } else {
16      session user => undef;
17      redirect uri_for 'login';
18    }
19  }
20 };
```

What's Happening Here?

- `database( 'registry' )` - named connection from config, always available
- `session->read( 'user' )` - is anyone logged in?
- Look them up in the registry
- `database({ ... })` - dynamic connection, per-request, per-user
- Both the user record and tenant DBH go into `var( )` for the rest of the request

If the user is gone or inactive: clear the session, redirect to login.

Dancer2::Plugin::Auth::Tiny

Requiring an authenticated user requires a simple route decorator:

```
1 get '/profile' => needs login => sub {  
2   my $user = var 'user';  
3   render( 'profile', { user => $user } );  
4 };
```

Unauthenticated requests are redirected to `/login` automatically.

Dancer2::Plugin::Auth::Tiny checks for a logged-in user via the session. We'll wire that up in the login route.

The Login Route

```
1 post '/login' => sub {
2   my $username = body_param( 'username' ) // '';
3   my $password = body_param( 'password' ) // '';
4
5   my $user = database( 'registry' )->quick_select(
6     'users', { username => $username }
7   );
8
9   if ( defined $user and verify_password( $password, $user->{ password } ) ) {
10    app->change_session_id;
11    session user => $username;
12    push_response_header 'HX-Redirect' => uri_for( '/' );
13  } else {
14    # timing attack mitigation: always run a password check
15    verify_password( 'foobar',
16      '$argon2id$v=19$m=65536,t=2,p=1$M/KquUs0lr1...$Zfnz...' )
17    unless defined $user and $user->{ is_active };
18
19    status 401;
20    return 'Invalid username or password.';
21  }
22 };
```

The Login Process

app->change_session_id prevents session fixation/replay attacks. Call it immediately after a successful authentication, before writing anything to the session.

Timing attack mitigation: Failure conditions resolve faster than a successful **verify_password()**. Always run the check, even for unknown users, so response time doesn't leak account existence.

push_response_header 'HX-Redirect' tells htmx to navigate the browser to the new URL rather than swapping content. No separate branch needed; htmx handles it client-side.

Injecting User Into Every Template

Rather than passing `user` explicitly in every route, use the `before_template_render` hook:

```
1 hook before_template_render => sub( $tokens ) {  
2     my $user = var 'user';  
3     $tokens->{ user } = $user;  
4 };
```

Every template now has `[% user.display_name %]` available automatically.

Logout

```
1 any '/logout' => sub {
2   my $username = session->read( 'user' );
3   if ( defined $username ) {
4     app->destroy_session;
5     info "User $username logged out successfully";
6   }
7   if ( request_header( 'HX-Request' ) ) {
8     push_response_header 'HX-Redirect' => uri_for( '/' );
9     return '';
10  }
11  redirect uri_for '/';
12 };
```

- `any` handles GET or POST; logout links and forms both work
- `app->destroy_session` is a full session wipe, not just clearing a key
- `htmx-aware`: sends `HX-Redirect` instead of a 302

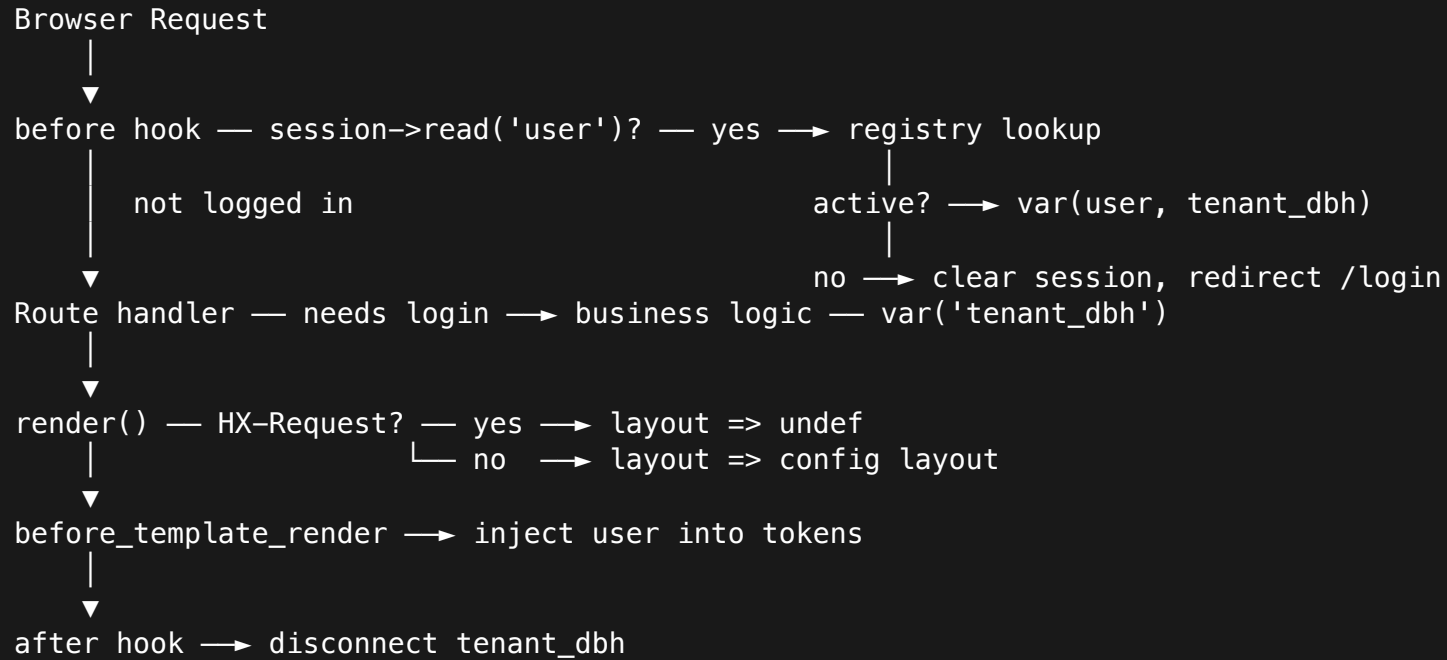
Clean Up, Clean Up, Everybody, Everywhere

```
1 hook after => sub {  
2   my $tenant_dbh = var 'tenant_dbh';  
3   if ( $tenant_dbh && $tenant_dbh->{ Active } ) {  
4     my $user = var 'user';  
5     debug 'Disconnecting from tenant ' . $user->{ username };  
6     $tenant_dbh->disconnect;  
7   }  
8 };
```

- Checks { Active } before disconnecting; connection may already be gone
- **Dancer2::Plugin::Database** manages the registry connection; the per-tenant connection is ours to clean up

Don't leave me (the tenant) hanging!

Putting It All Together



What We Covered

- **htmx basics** - HX-* headers, fragment responses
- **render()** helper - htmx detection in one place, routes stay clean
- **Multi-tenancy** - user-as-tenant, per-request SQLite connections via **database()**
- **before / after** hooks set up and tear down per-request
- **Dancer2::Plugin::Auth::Tiny** - easy authentication
- **Login** — **change_session_id**, timing attack mitigation, **HX-Redirect**
- **before_template_render** hook injects shared data without repeating yourself

Resources

- Dancer2: perldancer.org | CPAN: [Dancer2](#)
- htmx: htmx.org
- Dancer2::Plugin::Auth::Tiny: on CPAN
- Dancer2::Plugin::CryptPassphrase: on CPAN
- These slides + example app:
 - github.com/cromedome/dancer2-htmx-multitenant



Dancer2 makes multi-tenancy approachable.

htmx makes your users happy.

Your ops team will appreciate the lack of a build pipeline.

Questions?

Thank You

Jason Crome

- GitHub / LinkedIn: [cromedome](#)
- [crome-plated.com](#) / [cromedome.net](#)